

FAI Job Search Multi-Task Agent System

Project Architecture Analysis — Agents, Schemas, APIs, Academic Concept Mapping

1. Executive Overview

A layered, failover-resilient multi-agent architecture that converts an unstructured PDF resume into verified, ATS-optimized, job-specific PDF resumes. Every agent is a subclass of one LLM-orchestration base class, and every agent boundary is enforced by a typed Pydantic schema contract. The system targets students, freshers and job seekers who need resume variants tailored to many live job openings without ever fabricating a claim.

Six specialized agents execute in sequence, joined by a producer-critic reflection loop that audits every generated resume against the original ground truth before a PDF is ever compiled.

End-to-End Data Flow

```
PDF Resume
-> [A1] ResumeParserAgent    => CandidateProfile      (knowledge representation)
-> [A2] RoleRecommenderAgent => RoleRecommendationResponse (goal-state formulation)
-> [A3] JobSearchAgent       => List[JobListing]      (informed search + query expansion)
-> [A4] MatchGapAgent        => MatchReport           (heuristic ATS scoring + gap deduction)
-> [A5] TailorAgent          => TailoredResume        (grounded content synthesis)
-> [A6] VerifierAgent        => VerificationReport     (CSP grounding audit)
    reflection loop: [A5] <-> [A6] until APPROVED
-> ReportLab document engine => ATS-parseable PDF bytes
```

2. Layered Architecture

The codebase is organised into six layers with a strict one-way dependency rule: the UI depends on the orchestrator, the orchestrator depends on agents, and agents depend only on schemas and tools. Agents never import one another except through the orchestrator, which keeps each agent independently testable.

Layer	Files	Responsibility
UI	ui/app.py (689 lines)	Streamlit dashboard; five gated stages; session-state machine; 200 lines of injected custom CSS design system.
Orchestration	agents/tailoring_orchestrator.py	Producer-critic reflection loop; drives TailorAgent and VerifierAgent to convergence; streams status callbacks.
Agent Layer	agents/base_agent.py + 6 agent modules	Multi-provider LLM reasoning with structured schema-bound output and deterministic fallbacks.
Schema Layer	schemas/ (4 modules, 17 models)	Typed contracts and the candidate knowledge representation; every agent boundary validated by Pydantic v2.
Tool Layer	tools/ (6 modules)	PDF read/write, live HTTP job feeds, SQLite cache, relevance ranking.
Config	config/settings.py	Dotenv loading, provider key validation, model registries, retry and loop tuning constants.

3. The Six Agents

Each agent implements one stage of the pipeline, inherits the shared LLM transport from BaseAgent, and pairs its primary LLM path with a deterministic fallback so the system remains demonstrable with zero API keys configured.

Agent 1 - ResumeParserAgent (agents/resume_parser_agent.py)

Concept: Knowledge Representation & Ontological Engineering (Module VI)

Input: raw PDF text | Output: CandidateProfile

- Transforms unstructured resume prose into a typed ontology: CandidateContact, EducationItem, WorkExperienceItem, ProjectItem and the five-category TechnicalSkills matrix.
- Retains profile.raw_text, which becomes the immutable ground-truth reference later audited by Agent 6.
- Deterministic fallback _build_deterministic_profile() scans a 30-item skill catalogue with regex, detects fresher status and extracts the degree via pattern matching.

Agent 2 - RoleRecommenderAgent (agents/role_recommender_agent.py)

Concept: State Space Planning & Goal Formulation (Module VII)

Input: CandidateProfile | Output: RoleRecommendationResponse

- Formulates 3 to 5 target goal states; the schema itself enforces the range through min_length=1 and max_length=5.
- Each role carries fit_score (0-100), rationale, key_matching_strengths and recommended_search_keywords, which seed Agent 3's queries.
- Detects skill domains (is_data_ml, is_frontend, is_backend, is_cloud) to rank roles appropriately for the candidate's seniority.

Agent 3 - JobSearchAgent (agents/job_search_agent.py)

Concept: Uninformed vs. Informed Search and Query Expansion (Modules II & III)

Input: role title + location | Output: List[JobListing]

- Executes an agentic loop: initial query, then if zero results are returned it generates three to four alternative queries and retries the top two.
- Deduplicates across attempts using a normalised title_company key set.
- Owns a JobSearchTool instance and performs classic tool calling.

Agent 4 - MatchGapAgent (agents/match_gap_agent.py)

Concept: Heuristic Evaluation (Module III) and Deductive Inference (Module V)

Input: CandidateProfile + JobListing | Output: MatchReport

- Weighted ATS formula: overall = hard_skills x 0.50 + project_relevance x 0.30 + experience_education x 0.20.
- Bidirectional substring set intersection decides matched versus missing skills, so 'React' matches 'React.js'.
- Gap analysis is deductively inferred and surfaced as matched_skills, missing_skills and actionable tailoring_recommendations.

Agent 5 - TailorAgent (agents/tailor_agent.py)

Concept: State Space Content Synthesis (Module VII)

Input: profile + job + MatchReport (+ auditor feedback) | Output: TailoredResume

- System prompt encodes an explicit anti-hallucination policy: may rephrase, reorder and emphasise, but must never invent companies, degrees, years of experience or technologies.
- Accepts auditor_feedback and is contractually obliged to correct every flagged violation on the next pass.
- Computes ats_score_after = min(before + 18, 96) as the projected keyword-density gain.

Agent 6 - VerifierAgent (agents/verifier_agent.py)

Concept: Constraint Satisfaction (Module IV) and Adversarial Critic (Module IX)

Input: CandidateProfile + TailoredResume | Output: VerificationReport

- Enforces the CSP constraint: Tailored Resume content is a subset of Candidate Truth.
- Runs two constraint checks: skill subset inclusion yielding UNGROUNDED_SKILL, and company-name matching yielding FALSE_EXPERIENCE.
- Emits VerificationIssue records with issue_type, flagged_content, reason and suggested_correction, plus a binary APPROVED / REVISION_NEEDED status and a LOW / MEDIUM / HIGH hallucination risk level.

Orchestrator - TailoringOrchestrator

Manages the bidirectional feedback loop and enforces convergence before any document is compiled.

```
iteration = 0
while iteration <= max_retries (default 2):
    tailored = TailorAgent(profile, job, match_report, feedback) # produce
    audit = VerifierAgent(profile, tailored) # critic
    if audit.status == 'APPROVED': break # converge
    feedback = audit.feedback_for_tailor # reflect
```

Returns the tuple (TailoredResume, VerificationReport, iteration_count). A status_callback propagates each cycle into the Streamlit trace box so the reflection loop is visible to the user while it runs.

4. LLM Orchestration Layer - BaseAgent

agents/base_agent.py is the most technically substantial module. It provides structured prompt execution with automatic multi-provider failover so that free-tier quota exhaustion never crashes the pipeline.

Priority	Provider	Endpoint / Transport	Auth	Model Roster
1 (primary)	OpenRouter (free models)	openrouter.ai/api/v1/chat/completions via requests	Bearer sk-or-*	9 free models incl. qwen, gemma, nemotron
2 (secondary)	Groq (fast inference)	api.groq.com/openai/v1/chat/completions via requests	Bearer gsk_*	gpt-oss-120b, gpt-oss-20b, qwen
3 (tertiary)	Google Gemini	google-genai SDK models.generate_content	API key	gemini flash tier (4 candidates)

Resilience Mechanisms

- **Class-level circuit breaker.** `_tripped_providers` is a set shared by every agent instance for the lifetime of the process. A provider with permanently exhausted quota is disabled globally rather than retried per call.
- **Permanent versus transient quota detection.** `_is_permanent_quota_exhaustion()` matches body markers such as 'free-models-per-day', 'add 10 credits' and 'monthly limit'. Permanent exhaustion trips the breaker immediately; transient failures back off.
- **HTTP status triage.** 429 triggers exponential backoff ($4.0s \times 2^{(n-1)}$); 402 and 403 are treated as billing or auth faults and trip the breaker; 5xx retries; any other status advances to the next model.
- **Model rotation.** Each provider holds an ordered model list; the transport walks the list until one returns content that satisfies the schema.
- **Self-repair loop.** On Pydantic validation failure `_repair_and_validate()` sends the model its own invalid JSON, the truncated validation error and the JSON Schema, asking for a corrected object at temperature 0.0.
- **JSON sanitisation.** `_extract_json_string()` strips markdown code fences, then falls back to slicing the outermost brace pair by index.
- **Schema injection.** `response_schema.model_json_schema()` is embedded in every prompt for OpenAI-compatible providers; Gemini instead uses `response_mime_type='application/json'`.

5. Schema Layer - The Knowledge Representation

Schema Module	Models	Purpose
schemas/resume_schema.py	CandidateContact, EducationItem, WorkExperienceItem, ProjectItem, TechnicalSkills, CandidateProfile	The candidate ontology and single source of truth for all downstream agents.
schemas/job_schema.py	JobListing, JobSearchQuery, RoleRecommendation, RoleRecommendationResponse	Normalized job openings and the goal-state formulation contract.
schemas/analysis_schema.py	SkillMatchItem, MatchBreakdown, MatchReport	Heuristic scoring output with matched and missing skill sets.
schemas/tailoring_schema.py	TailoredBullet, TailoredExperience, TailoredProject, TailoredResume, VerificationIssue, VerificationReport	Tailored document contract plus the audit verdict contract.

Design Details

- Every field carries a `description=` argument. These descriptions are serialized into the LLM prompt as part of the JSON Schema, so field documentation doubles as prompt engineering.
- Validation constraints encode business rules directly: `ge=0` and `le=100` on all score fields, `min_length=1` and `max_length=5` on the recommended-roles list.
- `TailoredBullet` stores both `original_text` and `tailored_text`, giving a per-bullet audit trail that makes diffing and grounding checks possible.
- `TechnicalSkills.all_technical_skills()` performs order-preserving, case-insensitive deduplication across the five skill categories.
- `CandidateProfile.raw_text` preserves provenance of the original document for grounding validation.

6. Tool Layer and Live APIs

6.1 tools/pdf_reader.py - Resume Ingestion

- pypdf.PdfReader iterates pages, calling page.extract_text() and joining the results before clean_extracted_text() strips blank lines and normalizes whitespace.
- Accepts str, Path, BinaryIO or raw bytes, so it works both with disk files and with the Streamlit UploadedFile stream.
- File-handle hygiene: only streams the function opened itself are closed, enforced in a finally block.

6.2 tools/remote_feeds.py - Five Free Job Sources

Every source is free and requires no API key. Each fetcher is wrapped in its own try/except so a single failing source never blocks the others.

Source	Endpoint	Method	Notes
LinkedIn	/jobs-guest/jobs/api/seeMoreJobPostings/search	BeautifulSoup scrape	Guest endpoint, no auth. Selectors base-search-card__title, base-search-card__subtitle, job-search-card__location. Regex fallback when bs4 is absent. Strips query string from URLs; location-aware.
Remotive	remotive.com/api/remote-jobs?search=	JSON API	Official API, limit=25, filters on title and tags.
Jobicy	jobicy.com/api/v2/remote-jobs?tag=	JSON API	jobType and jobGeo may be str or list; defensively flattened.
RemoteOK	remoteok.com/api	JSON feed	data[1:] skips the metadata element; filters on position plus tags.
ArbeitNow	arbeitnow.com/api/job-board-api	JSON API	Reads payload['data']; maps candidate_required_location to location.

- Shared hardened defaults: a custom User-Agent, a (15, 25) connect/read timeout pair, and up to two retry attempts.
- An adapter layer (_normalise_remoteok, _normalise_arbeitznow, _normalise_remotive, _normalise_jobicy) maps every heterogeneous payload to one uniform internal dict with id, title, company, location, url, description, source, employment_type and required_skills.
- uuid.uuid4() supplies a fallback identifier whenever the upstream feed omits one.

6.3 tools/job_search_tool.py - Hybrid Retrieval and Relevance Ranking

- **Step 1.** Query the SQLite cache first via get_recent() with a configurable max_age_hours (default 24).
- **Step 2.** On a cache miss, _fetch_from_feeds() merges results from all five sources.
- **Step 3.** Persist everything with bulk_upsert() so subsequent searches avoid the network.
- **Step 4.** Rank and filter through the weighted relevance heuristic described below.

Weighted relevance heuristic (Module III):

- Title token match scores x3.0, required-skills match x2.0, description match x1.0, and a full-phrase-in-title bonus adds x4.0.
- _query_tokens() strips a 40-word stopword set covering seniority words, role nouns, employment types and Indian city names.
- Ranking uses a stable sort on (-score, original_index), so ties preserve original feed order and the UI never reshuffles unpredictably.
- If nothing scores above zero the unranked feed order is returned, so the dashboard never renders an empty stage.
- Additional APIs: refresh_cache() forces a bypass of the cache, search_jobs_by_date_range() filters cached rows by an exact UTC window, and cache_stats() reports counters.

6.4 tools/job_cache.py - SQLite Informed-Search State Store

- A single jobs table with id as TEXT PRIMARY KEY, plus two indexes: idx_jobs_title on LOWER(title) and idx_jobs_fetched_at on fetched_at.
- Writes use INSERT ... ON CONFLICT(id) DO UPDATE so re-fetching a job refreshes it in place and stamps CURRENT_TIMESTAMP.
- _as_text() coercion solves a concrete defect: public feeds deliver the same logical field as str, list, dict or None, and sqlite3 refuses to bind list or dict parameters, which previously caused silent per-record write failures.
- Reads build an AND-across-tokens LIKE filter over title, required_skills and description; the location filter is skipped for the values remote, any and all.
- clear_stale(older_than_hours=72) provides TTL cleanup.

6.5 tools/pdf_generator.py - ATS Document Engine

- ReportLab SimpleDocTemplate on Letter with 0.5in margins, rendered into a BytesIO buffer so the function returns bytes and can optionally write to disk.
- ATS-safe choices: a single column, base-14 Helvetica and Helvetica-Bold fonts (no custom embedded fonts, which is critical for reliable ATS text extraction), no content tables, no images and no text boxes.
- Seven-section flow: header with name and pipe-joined contact details plus an HRFlowable rule, professional summary, technical skills matrix, work experience, key projects, education and certifications.

- Six custom ParagraphStyles use a restrained hex palette (#1A202C, #2B6CB0, #2D3748) with keepWithNext on headings to prevent orphaned section titles.
- Bullets use a hanging indent (leftIndent=14, firstLineIndent=-10) and only minimal inline markup: b, i and font color.

6.6 tools/generate_workflow_pdf.py - Documentation Renderer

A separate 302-line utility that renders docs/System_Architecture_and_Workflow.pdf using Table, TableStyle and KeepTogether to lay out architecture diagrams.

7. Configuration and Tuning (config/settings.py)

A module-level singleton settings object loads the .env file via load_dotenv() and calls ensure_directories() at import time, so runtime directories always exist.

Setting	Value	Rationale
TEMPERATURE	0.2	Low temperature chosen for factual, deterministic resume tailoring.
MAX_TOKENS	8192	Room for large multi-section JSON responses.
MAX_RETRIES_PER_MODEL	3	Bounded retries before advancing to the next model.
RETRY_BACKOFF_SECONDS	4.0	Base for exponential backoff ($\times 2^{(n-1)}$).
MAX_VERIFICATION_RETRIES	2	Maximum reflection cycles in the tailoring loop.
MIN_ACCEPTABLE_ATS_SCORE	65	Threshold separating acceptable from weak matches.
Provider key validation	sk-or- and gsk_ prefixes	Prefix checks reject malformed or placeholder keys before any request is attempted.
Model registries	Lists in settings.py	Free-tier models rotate frequently, so the active roster lives in config, not in code paths.

Declared project paths: BASE_DIR, DATA_DIR, OUTPUT_DIR and SQLITE_DB_PATH (data/job_cache.db).

8. UI Layer - Streamlit Dashboard (ui/app.py)

Five gated stages, each revealed only when the preceding session-state key exists, which naturally enforces pipeline ordering in the UI.

Stage	Agents	Renderers
1. Resume Ingestion	Agent 1	PDF uploader, parsed profile expander, verified skill pills.
2. Career Strategy	Agent 2	Role cards with conic-gradient fit rings, rationale and selectable targets.
3. Job Discovery + Gap Analysis	Agents 3 and 4	Job cards with per-source badges, ATS score rings, Apply links, matched and missing skill pills, tailoring recommendations.
4. Autonomous Tailoring and Audit	Agents 5 and 6	Live multi-agent execution trace fed by status_callback, plus a st.progress bar.
5. PDF Delivery	Document engine	Download buttons, before/after ATS delta badges, audit verdict and risk level.

Interface Design System

- Roughly 200 lines of injected CSS implementing a dark theme: Inter and JetBrains Mono webfonts, radial-gradient backgrounds and glassmorphism cards.
- An animated hero banner uses a rotating conic-gradient pseudo-element and a pulsing badge dot.
- The pipeline tracker renders five steps with step-done, step-active and step-pending states and gradient connectors.
- Source badges are colour-coded per portal: LinkedIn blue, Remotive green, Jobicy violet, RemoteOK amber, ArbeitNow red.
- Skill pills exist in three variants: default, matched (green) and missing (red).
- The trace box is styled as a terminal to make the reflection loop legible during a demo.

State and Fault Handling

- Session state keys: resume_text, profile, role_recommendations, selected_role, discovered_jobs, match_reports, tailored_resumes and generated_pdfs.
- match_reports, tailored_resumes and generated_pdfs are dictionaries keyed by job id, which enables batch tailoring of several jobs in a single pass.
- Uploading a new resume automatically invalidates every downstream stage.
- Stage 3 wraps each per-job analysis in its own try/except so one failed job reports a quota exhaustion message instead of aborting the whole batch.
- The sidebar exposes a cache refresh control, a full pipeline reset, and live cache counters.

9. Academic Concept Mapping

Mapping each agent and tool onto the corresponding Foundation of Artificial Intelligence syllabus module.

Syllabus Module	Concept	Implementation
II and III - Search	Uninformed and informed search, query expansion	JobSearchAgent._generate_query_expansions() rewrites queries on sparse results; JobCache acts as a persistent informed-search state store.
III - Heuristic Evaluation	Weighted heuristic ranking and scoring	_relevance_score() in the search tool; the 50/30/20 weighted ATS formula in MatchGapAgent.
IV - Constraint Satisfaction	CSP on truthfulness	VerifierAgent enforces Tailored Resume content is a subset of Verified Profile Truth, checked both by LLM audit and by a deterministic heuristic.
V - Inference and Logic	Deductive gap analysis	Logical deduction of missing mandatory skills and contextual tailoring recommendations in MatchGapAgent.
VI - Knowledge Representation	Ontological resume representation	The CandidateProfile Pydantic ontology structuring raw unstructured PDF text, with raw_text retained as provenance.
VII - Planning	State space planning and goal formulation	RoleRecommenderAgent maps capabilities to three to five optimal career goal states; TailorAgent performs content-space synthesis.
IX - Multi-Agent Systems	Producer-critic reflection loop	TailoringOrchestrator runs the TailorAgent / VerifierAgent feedback convergence loop until the audit is APPROVED.

10. Technology Stack

Concern	Technology	Version / Endpoint
LLM provider (primary)	OpenRouter free model router	REST, openrouter.ai/api/v1
LLM provider (secondary)	Groq (gpt-oss-120b / 20b)	REST, OpenAI-compatible
LLM provider (tertiary)	Google Gemini via google-genai	>= 1.0.0
Schema validation	Pydantic v2	>= 2.6.0
Configuration	python-dotenv	>= 1.0.0
PDF parsing	pypdf	>= 4.0.0
PDF generation	ReportLab Platypus	>= 4.1.0 (installed 5.0.1)
HTTP client	requests	>= 2.31.0
HTML scraping	BeautifulSoup4 with regex fallback	>= 4.12.0
Cache / state store	sqlite3 (standard library)	data/job_cache.db
Dashboard	Streamlit	>= 1.32.0

11. Design Patterns

Pattern	Where It Appears
Provider failover with circuit breaker	BaseAgent._tripped_providers

Pattern	Where It Appears
Retry with exponential backoff	BaseAgent._chat_completion_with_retry
Producer-critic (reflection loop)	TailoringOrchestrator
Adversarial critic / self-verification	VerifierAgent
Strategy (deterministic fallback pairing)	Each agent pairs an LLM path with a _build_deterministic_* or heuristic method
Cache-aside (repository)	JobCache and JobSearchTool
Observer (callback)	status_callback streaming into the Streamlit trace box
Adapter / normalizer	The five _normalise_* feed functions in remote_feeds.py
Schema as contract	Pydantic models enforcing every agent boundary
Singleton	config.settings.settings

12. Engineering Highlights and Known Gaps

Highlights

- Zero-key demonstrability: every agent ships a deterministic fallback, so the full pipeline runs offline with no LLM provider configured.
- Three-provider failover with a process-wide circuit breaker and permanent-versus-transient quota classification.
- Model self-repair: a schema violation is fed back to the same model as a corrective task.
- Provenance and auditability: raw_text ground truth, original_text versus tailored_text pairing, and structured VerificationIssue records.
- Graceful degradation: the LinkedIn scraper falls back to regex when BeautifulSoup is unavailable, and every feed is individually fault-isolated.
- Determinism where it matters: temperature 0.2 plus stable sorting for reproducible output.
- Real defect resolved defensively: the SQLite _as_text() coercion for heterogeneous feed payload types.

Known Gaps Between README and Code

- README references tests/test_pipeline.py and tools/mock_job_data.py; neither exists in the repository.
- README and .env.example document SerpAPI and RapidAPI keys, but the implemented tool layer uses the five key-free public feeds instead.
- USE MOCK SEARCH is present in .env but is no longer read anywhere in the code path.
- The documented 6-stage pipeline is presented in the dashboard as 5 UI stages, because gap analysis shares a stage with job discovery.